

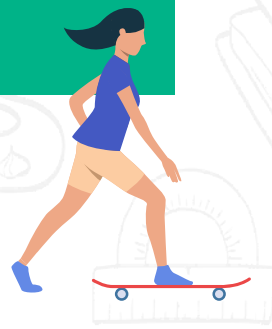
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 3-1

1	2	3	4
Shell Programming	~	~	~

Objectives

1、了解 Shell 脚本执行方法，掌握 Shell 环境变量的使用方法

2、理解输出重定向使用方法，了解管道使用方法



Pre-assessment

1、Shell 中如何定义变量?

2、是否了解环境变量的概念，如何配置环境变量?

3、是否了解输出/输出重定向和管道功能?

4、了解 Linux 中归档和压缩的概念?

5、Linux 中如何进行归档?



1

Shell Programming

1.1 Shell Programming

1.1.1 Linux shell

- Linux Shell 主要提供以下两种功能：
  - 解释用户在命令行提示符下输入的命令
  - 编写 Shell 脚本，实现高级管理功能
- Shell 两种执行命令的方式：
  - 第一种，交互式。用户每输入一条命令，Shell 就解释执行一条
  - 第二种，批处理式。需要事先编写一个 Shell 脚本，其中包含若干条命令，让 Shell 一次将这些命令执行完，编写 Shell 脚本的过程就是 Shell 编程

备注：Shell 脚本是解释执行的，不需要编译，Shell 程序从脚本中一行一行读取并执行这些命令，相当于一个用户把脚本中的命令一行一行敲到 Shell 提示符下执行



1.1.2 Shell script

- Shell 脚本是一个文本文件，Shell 脚本编程无需编译器，也不需要集成开发环境，一般使用文本编辑器即可，首选的编辑器是 Vim 或 Emacs，在桌面环境中可直接使用图形化编辑器 gedit 或 kate

- Shell 脚本编写的“Hello world”：

```
$ sudo touch hello
$ sudo gedit hello
```

```
#！ /bin/bash
# 显示"Hello World! ”
echo "Hello World!"
```



- 注解：
  - “#!“ 指定 Shell 脚本的运行环境，声明该脚本使用哪个 Shell 程序运行；如果没有指定，则以当前正在执行的 Shell 来解释执行
  - 以“#”开头的行是注释行，Shell 在执行时会忽略“#”之后的所有内容
  - echo 命令用来显示字符串信息

1.1.2 Shell script

- Shell 脚本可以包含外部脚本，将外部脚本的内容合并到当前脚本。包含外部脚本文件的用法如下：
  - . 脚本文件名
  - source 脚本文件名

  备注：两种方式的作用一样，注意点号和脚本文件名之间一定要有一个空格

- 执行 Shell 脚本的方式有 3 种：

（1）在命令提示符下直接执行

- 与执行一般的可执行文件的方式相似，首先赋予 Shell 脚本执行权限(sudo chmod +x 脚本文件)，然后在终端中直接执行
- 注意：如果像命令那样直接输入脚本文件名，还需要让该脚本所在的目录被包含在环境变量PATH所定义的命令搜索路径中，否则就要明确指定脚本文件的路径



1.1.2 Shell script

- 举例：编写上述 hello 的 Shell 脚本，执行脚本：

```
$ ./hello
```

  - 执行脚本命令时在脚本文件名前加上了“./”，表明启动当前目录下的脚本文件 hello
  - 如果直接执行脚本文件 hello，Linux 系统会到命令搜索路径（PATH）中去查找该脚本文件，由于此例脚本位于用户主目录，会提示指定文件不存在

(2) 在指定的 Shell 下执行脚本

- 可以在指定的 Shell 下执行脚本，以脚本名作为参数。基本用法如下：

  Shell 名称 脚本名 [参数]

- 举例：执行 hello 脚本

```
$ bash ./hello      # 使用bash
$ sh ./hello        # 使用sh
```



(3) 将输入重定向到 Shell 脚本

1.1.3 Shell variables

Linux 的 Shell 编程支持以下3种变量类型：(1)用户自定义变量、(2)环境变量、(3)系统变量

(1) 用户自定义变量

- 编写 Shell 脚本时定义的变量，可在 Shell 程序内任意使用和修改，可看作局部变量，仅在当前 Shell 实例中有效
- 变量定义：在 Shell 编程中，变量是非类型性质的，不必指定变量是数字，还是字符串，给变量赋值的过程也就是定义一个变量的过程，其格式如下：

  变量名=值

  - 备注：在赋值符号两边不允许有空格。如果值中含有空格、制表符或换行符，则要将这个字符串用引号括起来
  - 在同一个变量中，可以一次存放整型值，下一次再存储字符串
  - 举例：

```
$ x=8
$ x=” Hello”
```



1.1.3 Shell variables

- 变量名的命名应当遵循如下规则：
  - 首个字符必须为字母（a ~ z, A ~ Z ）
  - 中间不能有空格，可以使用下画线（_）
  - 不能使用标点符号
  - 不能使用 Shell 中的关键字（在 bash 中可用 help 命令查看保留关键字)
- 删除变量：使用 unset 命令可以删除变量，语法如下：

  unset 变量名

  举例：

```
$ unset x
```



1.1.3 Shell variables

- 变量引用：
  - 如果要引用变量值，可以在变量名前面加一个美元符号“$”。例如变量名为myName, 使用$myName 就可以引用该变量
- 变量显示：
  - 通常使用函数 echo 来显示变量，例如：

```
# 将一个字符串赋值给变量 hello
$ hello=” Hello World!”
# 显示变量 hello 的值：
$ echo $hello
```

  - 通过 `echo $PATH` 可以查看 PTAH 变量的路径



1.1.3 Shell variables

(2) 环境变量

- Linux 中环境变量包括 **系统级** 和 **用户级**，系统级的环境变量是每个登录到系统的用户都要读取的系统变量，用户级的环境变量则是该用户使用系统时加载的环境变量。为了与普通变量进行区分，通常将**环境变量名设为大写**

- **1) 系统级**：
  - /etc/environment：Linux 系统在登录时读取的第一个文件，用于为所有进程设置环境变量。系统使用此文件时并不是执行此文件中的命令，而是根据 KEY=VALUE 模式的代码，对 KEY 赋值以 VALUE。文件中如果要定义 PATH 环境变量，只需加入一行形如 PATH=$PATH:/xxx/bin 的代码即可
  - /etc/profile：Linux 系统登录时执行的第二个文件，可以用于设定针对全系统所有用户的环境变量。该文件一般是调用 /etc/bash.bashrc 文件



1.1.3 Shell variables

- **1) 系统级：**

  - /etc/bash.bashrc：系统级的 bashrc 文件，为每一个运行 bash shell 的用户执行此文件。此文件会在用户每次打开 shell 时执行一次

  - 注意：/etc/environment 是设置整个系统的环境，而 /etc/profile 是设置所有用户的环境，前者与登录用户无关，后者与登录用户有关。 这两个文件修改后一般都要重启系统才能生效

- **2) 用户级：**

  - ~/.profile：对应当前登录用户的 profile 文件，用于定制当前用户的个人工作环境。每个用户都可使用该文件输入专用于自己使用的 shell 信息。当用户登录时，该文件仅仅执行一次。默认情况下，其设置一些环境变量，执行用户的 .bashrc 文件



1.1.3 Shell variables

- **2) 用户级：**
  - **~/.bashrc：对应当前登录用户的 bash 初始化文件，当用户每次打开 shell 时，系统都会执行此文件一次。ROS使用过程中要设置此文件配置环境变量**
  - **bashrc 文件注解：Linux 系统中很多 shell，包括bash，sh，zsh，dash 和 korn 等，不管哪种 shell 都会有一个 .bashrc 的隐藏文件，它就相当于 shell 的配置文件。主要保存用户自定义环境变量、个性化设置信息等**

根据以上描述，这几个文件的执行先后顺序应当是：

1. **/etc/enviroment –>/etc/profile –>/etc/bash.bashrc**
2. **~/.profile –> ~/.bashrc**



1.1.3 Shell variables

➢ **3) 配置环境变量：**

- 1) 在 Linux 下设定环境变量时，如果只是临时用一下，可以直接在 shell 下用 set 或 export 命令设定环境变量。该操作只能在当前 shell 脚本下可用，切换到另一个终端就会失效，举例：

```
$ export PYTHONPATH=/home/yanting/learning/ssd-caffe/python
```

- 2) 如果希望此环境变量每次开机或打开 shell 时自动设定而无须每次都手动设定，那么需要将 export 命令写入以上的系统文件中，举例：

```
$ sudo gedit ~/.bashrc
$ export PYTHONPATH=/home/yanting/learning/caffe/python:$PYTHONPATH
$ source ~/.bashrc
```

注解：PATH采用:来分隔，冒号左右不需要空格；$PYTHONPATH表示变量引用



1.1.3 Shell variables

(3) 系统变量

- Linux 系统变量是系统为了正常执行命令，维持正常运转而自带的变量，常用**大写字母**表示，常见的 Linux 环境变量如下：

变量名	作用
HOME	表示当前用户的家目录
SHELL	表示当前用户的Shell解释器
LANG	表示系统的语言
RANDOM	调用该变量可以由系统生成一个随机数字
PATH	定义Shell解释器搜索命令的路径



1.2 Redirection and pipe

1.2.1 Redirection

- stdin，stdout 和 stderr：

  - 在Linux下，当一个用户进程被创建时，系统会自动为该进程创建三个数据流，即：stdin，stdout 和 stderr

    (1) 三个数据流默认是表现在用户终端上，执行一个 shell 命令行时通常会自动打开三个标准文件：

    - 标准输入文件（stdin），通常对应键盘
    - 标准输出文件（stdout）和 标准错误输出文件（stderr），这两个文件都对应屏幕

    (2) 作用：将从标准输入文件中得到输入数据，正常输出数据输出到标准输出文件，将错误信息输出到标准错误文件中

    (3) 存在问题：

    - 输入数据从键盘输入时，输入的数据只能用一次，且输入有误修改起来不方便
    - 输出到屏幕上的信息只能看不能修改，无法对此输出作更多处理



1.2 Redirection and pipe

1.2.1 Redirection

- **输入/输出重定向：**

  为了解决上述问题，Linux 系统为输入、输出的传送引入了另外两种机制，即 输入/输出重定向 和 管道

  (1) 输入重定向：把命令（或可执行程序）的标准输入重定向到指定的文件中。也就是说，输入不来自键盘，而来自一个指定的文件

  - **命令<文件，表示将指定文件作为命令的输入**
  - **命令<<分界符，表示从标准输入文件（键盘）中读入，直到遇到分界符才停止（读入的数据不包括分界符），这里的分界符是自定义的字符串**
  - **备注：许多命令都支持输入重定向，此处使用命令指代各个支持输入重定向的命令**
  - **几个基本符号及其含义：0 表示stdin标准输入；1 表示stdout标准输出；2 表示stderr标准错误**



1.2 Redirection and pipe

1.2.1 Redirection

- 输入/输出重定向：

举例1：默认情况下，cat 命令会接受标准输入文件（键盘）的输入，并显示到标准输出（屏幕）。但如果用文件代替键盘作为标准输入，那么该命令会以指定的文件作为输入，并将文件中的内容读取并显示到屏幕

```
qlf@vmbox:~$ cat hello
#! /bin/bash
# 显示"Hello World! ”
echo "Hello World!"
```

举例2：输入 cat 命令后回车，cat 命令会默认捕获键盘的输入，并将捕获到的内容直接输出到屏幕(使用ctrl+c停止捕获)

```
qlf@vmbox:~$ cat
ros
ros
ubuntu
ubuntu
c++
c++
```



```
$ cat << aaa
```

```
qlf@vmbox:~$ cat << aaa
> ros
> ubuntu
> c++
> aaa
ros
ubuntu
c++
```

输入更改为 cat << aaa，cat 命令依旧会捕获键盘的输入，但遇到分界符aaa后会停止捕获，并将捕获的结果一次性的显示在屏幕上



1.2 Redirection and pipe

1.2.1 Redirection

- **输入/输出重定向：**

(2) 输出重定向：把命令（或可执行程序）的标准输出或标准错误输出重新定向到指定文件中。这样，该命令的输出就不显示在屏幕上，而是写入到指定文件中

- **命令>文件，将命令执行的标准输出结果重定向输出到指定的文件中，如果该文件已包含数据，会清空原有数据，再写入新数据**
- **命令2>文件，将命令执行的错误输出结果重定向到指定的文件中，如果该文件中已包含数据，会清空原有数据，再写入新数据**
- **命令>>文件，将命令执行的标准输出结果重定向输出到指定的文件中，如果该文件已包含数据，新数据将追加入到原有内容的后面**
- **命令2>>文件，将命令执行的错误输出结果重定向到指定的文件中，如果该文件中已包含数据，新数据将追加到原有内容的后面**



1.2 Redirection and pipe

1.2.1 Redirection

- **输入/输出重定向：**

举例：新建一个包含有 "Linux" 字符串的文本文件 Linux.txt，以及空文本文件 demo.txt，执行如下命令：

```
$ cat Linux.txt > demo.txt
$ cat demo.txt
        $ Linux
```

```
$ cat Linux.txt >> demo.txt
$ cat demo.txt
        $ Linux
        $ Linux
```



1.2 Redirection and pipe

1.2.1 Redirection

- 输入/输出重定向：

举例：新建一个包含有 "Linux" 字符串的文本文件 Linux.txt，以及空文本文件 demo.txt，执行如下命令：

```
$ cat b.txt > demo.txt
        $ cat: b.txt: No such file or directory  <-- 错误输出信息依然输出到了显示器中
```

```
$ cat b.txt 2> demo.txt
$ cat demo.txt
        $ cat: b.txt: No such file or directory  <--清空文件，再将错误输出信息写入到该文件中
```

```
$ cat b.txt 2>> demo.txt
$ cat demo.txt
        $ cat: b.txt: No such file or directory
        $ cat: b.txt: No such file or directory  <--追加写入错误输出信息
```

1.2 Redirection and pipe

1.2.2 Pipe

- 管道：管道是一种通信机制，通常用于进程间的通信。它表现出来的形式是将前面每一个进程的输出（stdout）直接作为下一个进程的输入（stdin）
- 管道又分为 匿名管道 和 具名管道，我们在使用过滤程序时经常会用到匿名管道，在命令行中由“|” 分隔符表示，其语法格式为：

  command1 | command2

  command1 | command2 [| commandN...]

  - 当在两个命令之间设置管道时，管道符 | 左边命令的输出就变成了右边命令的输入
  - 只要第一个命令向标准输出写入，第二个命令是从标准输入读取，那么这两个命令就可以形成一个管道
  - 管道可以和重定向连用



Post-assessment

1. 执行 Shell 脚本有哪几种方式?
2. 命令<文件 1>文件 2，表示什么意思?
3. 输出重定向的作用是什么?
4. 重定向与管道有什么区别?

主观题 10分

设置

1. 执行 Shell 脚本有哪几种方式?
2. 命令<文件 1>文件 2，表示什么意思?
3. 输出重定向的作用是什么?
4. 重定向与管道有什么区别?

正常使用主观题需2.0以上版本雨课堂

作答

ROS Summary

1. Shell Programming

a) Shell脚本：编写和执行脚本方法 

b) Shell变量：

① 用户自定义；

② 环境变量（系统级、用户级） 

c) 输出/输出重定向

d) 管道

Q&A

今日事今日毕

Question?



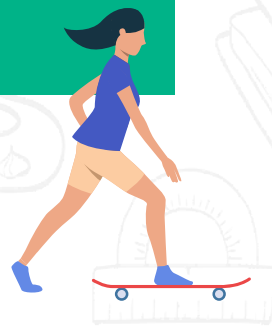
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 3-2

1	2	3	4
Archiving and Compression	~	~	~

Objectives

1、了解归档和压缩单独使用时常用命令的用法

2、掌握归档和压缩结合使用时4条命令的用法



Pre-assessment

1、了解 Linux 中归档和压缩的概念?

2、Linux 中如何进行归档?



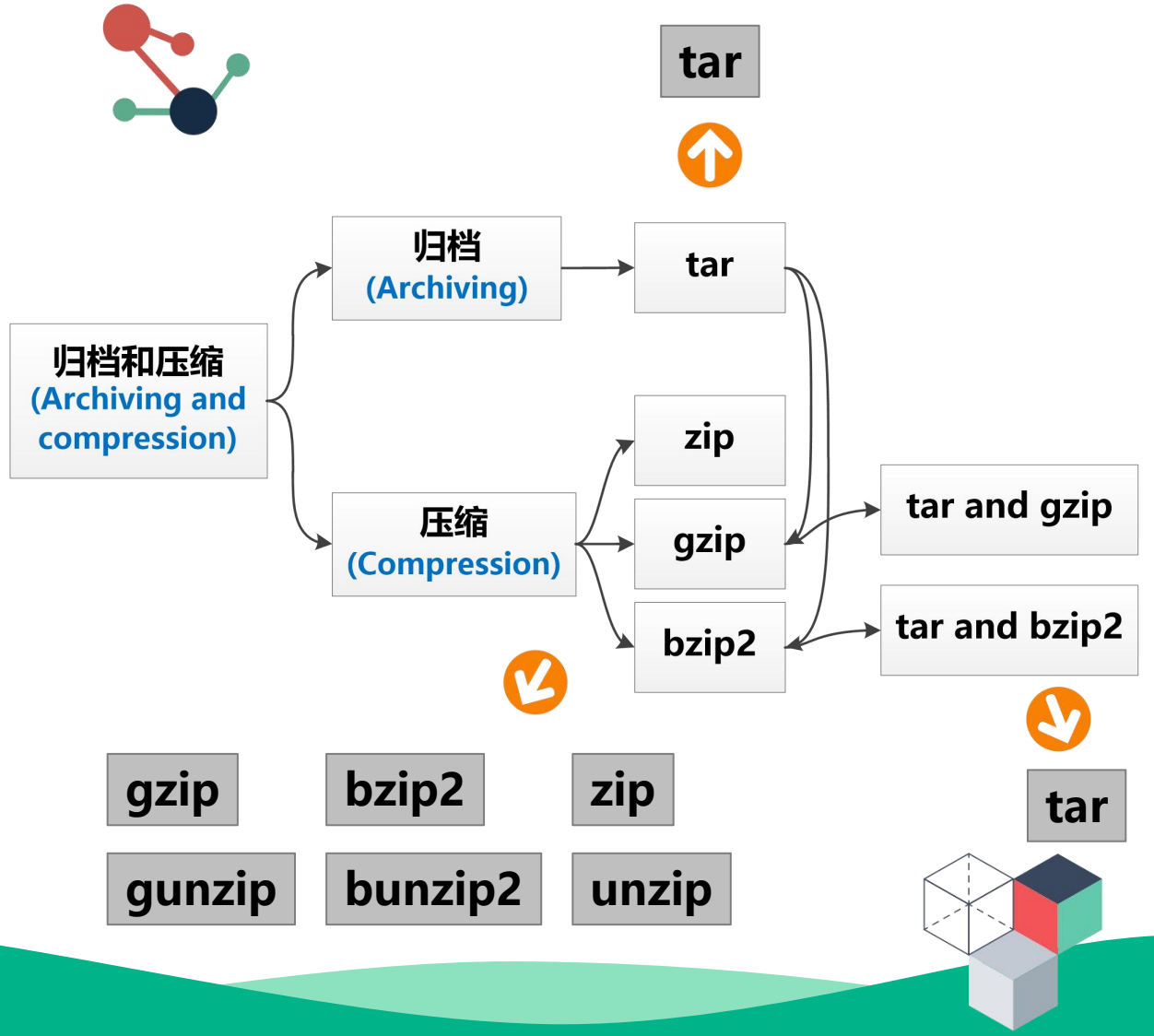
1

Archiving and Compression

1.1 Knowledge architecture

1.1.1 Overview

- 归档(Archiving)，也称打包，是指将一堆文件或目录打包成一个总文件，归档文件没有经过压缩，它占用的空间是其中所有文件和目录的总和
- 压缩(Compression)，是指将一堆文件或目录压缩成一个总文件，压缩过程采用了压缩算法，使得总文件所占用的磁盘空间比原文件的总和要小
- Linux中，归档工具为 tar，压缩工具有 zip、gzip、bzip2等



1.2 Archiving with tar

1.2.1 tar

- Linux 中最常用的归档（打包）命令是 tar，该命令可用于打包和解打包

1. 当 tar 命令用于打包操作时，命令格式为：

   tar [选项] 源文件或目录

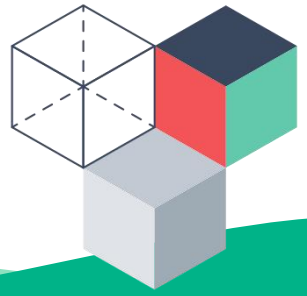
- 参数说明

  -c：将多个文件或目录进行打包

  -v：显示打包文件过程

  -f 包名：指定包的文件名

  -A：追加 tar 文件到归档文件



1.2 Archiving with tar

1.2.1 tar

- 举例说明：

✓ 例一：指定包名

```
$ tar -cvf anaconda-ks.cfg.tar anaconda-ks.cfg
```

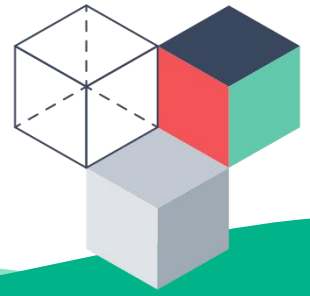
把 anaconda-ks.cfg打包为anaconda-ks.cfg.tar文件

选项 "-cvf" 一般是习惯用法，打包时需要指定打包之后的文件名，而且要用 ".tar" 作为扩展名

✓ 例二：打包多个文件

```
$ tar -cvf ana.tar anaconda-ks.cfg /tmp/
```

把 anaconda-ks.cfg文件和/tmp目录打包成ana.tar文件包



1.2 Archiving with tar

1.2.1 tar

2. 当 tar 命令用于解打包操作时，命令格式为：

   tar [选项] 压缩包

- 参数说明

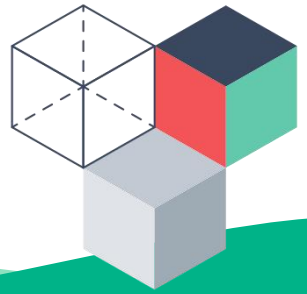
  -x：对 tar 包做解打包操作

  -v：显示解打包的具体过程

  -f：指定要解压的 tar 包的包名

  -t：只查看 tar 包中有哪些文件或目录，不对 tar 包做解打包操作

  -C 目录：指定解打包位置



1.2 Archiving with tar

1.2.1 tar

- 举例说明：

✓ 例一：

```
$ tar -xvf anaconda-ks.cfg.tar
```

解打包到当前目录

✓ 例二：

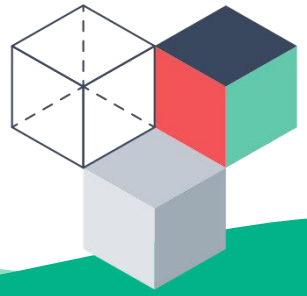
```
$ tar -xvf test.tar -C /tmp
```

把文件包 test.tar 解打包到 /tmp/目录下

✓ 例三：

```
$ tar -tvf test.tar
```

只查看文件包中有哪些文件



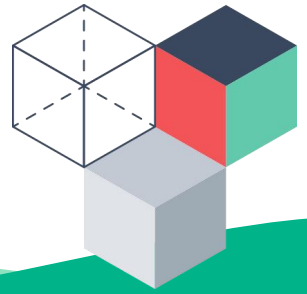
1.3 Compression

1.3.1 gzip

- gzip 是 Linux 系统中经常用来对文件进行压缩和解压缩的命令，通过此命令压缩得到的新文件，其扩展名标记为“.gz”
  - 备注：gzip 命令只能用来压缩文件，不能压缩目录，即使指定了目录，也只能压缩目录内的所有文件
  - gzip 命令的基本格式如下：

    gzip [选项] 源文件

    备注：命令中的源文件，当进行压缩操作时，指普通文件；当进行解压缩操作时，指压缩文件



1.3 Compression

1.3.1 gzip

- 参数说明：

  -c：将压缩数据输出到标准输出中，并保留源文件

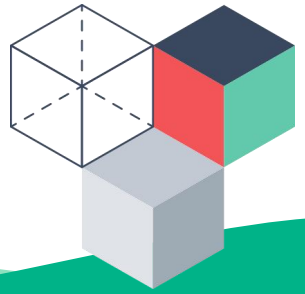
  -d：对压缩文件进行解压缩

  -r：递归压缩指定目录下以及子目录下的所有文件

  -v：对于每个压缩和解压缩的文件，显示相应的文件名和压缩比

  -l：对每一个压缩文件，显示以下字段：压缩文件的大小；未压缩文件的大小；压缩比；未压缩文件的名称

  -数字：用于指定压缩等级，-1 压缩等级最低，压缩比最差；-9 压缩比最高。默认压缩比是 -6



1.3 Compression

1.3.1 gzip

- **举例说明**:

✓ **例一：基本压缩，不保留源文件**

```
$ gzip install.log
```

压缩 instal.log 文件

✓ **例二：保留源文件压缩**

```
$ gzip -c anaconda-ks.cfg>anaconda-ks.cfg.gz
```

压缩文件的同时不删除源文件

✓ **例三：压缩目录**

```
$ gzip -r test/
```

gzip命令不会打包目录，

把test目录下所有的子文件分别压缩



1.3 Compression

1.3.2 gunzip

- gunzip 是一个使用广泛的解压缩命令，它用于解压被 gzip 压缩过的文件（扩展名为 .gz）
  - 备注：对于解压被 gzip 压缩过的文件，还可以使用 gzip 命令，即 gzip -d 压缩包
- gunzip 命令的基本格式为：

  gunzip [选项] 文件

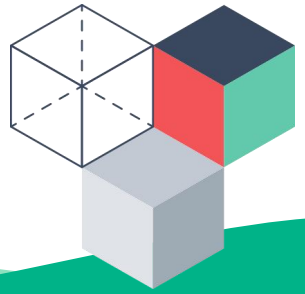
  - 参数说明：

    -r：递归处理，解压缩指定目录下以及子目录下的所有文件

    -c：把解压缩后的文件输出到标准输出设备

    -f：强制解压缩文件，不理会文件是否已存在等情况

    -l：列出压缩文件内容



1.3 Compression

1.3.2 gunzip

-v：显示命令执行过程

-t：测试压缩文件是否正常，但不对其做解压缩操作

- 举例说明：

✓ 例一：

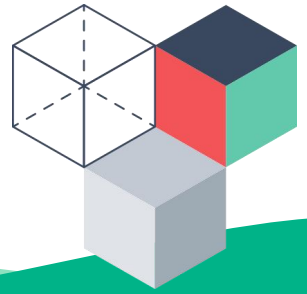
```
$ gunzip install.log.gz
```

直接解压缩文件

✓ 例二：

```
$ gunzip -r test/
```

解压目录下的内容



1.3 Compression

1.3.3 bzip2

- bzip2 命令同 gzip 命令类似，只能对文件进行压缩（或解压缩），对于目录只能压缩（或解压缩）该目录及子目录下的所有文件
- bzip2 命令的基本格式如下：

  bzip2 [选项] 源文件

- 参数说明：

  -d：执行解压缩，此时该选项后的源文件应为标记有 .bz2 后缀的压缩包文件

  -k：bzip2 在压缩或解压缩任务完成后，会删除原始文件，若要保留原始文件，可使用此选项

  -f：bzip2 在压缩或解压缩时，若输出文件与现有文件同名，默认不会覆盖现有文件，若使用此选项，则会强制覆盖现有文件

  -t：测试压缩包文件的完整性



1.3 Compression

1.3.3 bzip2

-v：压缩或解压缩文件时，显示详细信息

-数字：用于指定压缩等级，-1 压缩等级最低，压缩比最差；-9 压缩比最高

备注：gzip 不打包目录，但是如果使用“-r”选项，则可以分别压缩目录下的每个文件；bzip2 命令不支持压缩目录，没有“-r”选项

- 举例说明：
  - ✓ 例一：直接压缩文件

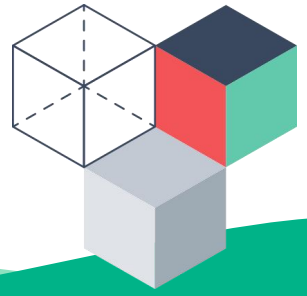
```
$ bzip2 anaconda-ks.cfg
```

将anaconda-ks.cfg文件压缩成".bz2"格式

  - ✓ 例二：压缩的同时保留源文件

```
$ bzip2 -k install.log.syslog
```

压缩并保留源文件



1.3 Compression

1.3.4 bunzip2

- 解压“.bz2”格式的压缩包文件，除了使用“bzip2 -d 压缩包名”命令外，还可以使用 bunzip2 命令
- bunzip2 命令的基本格式如下：

  bunzip2 [选项] 源文件

- 参数说明：

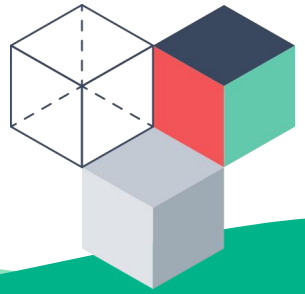
  -k：解压缩后，默认会删除原来的压缩文件。若要保留压缩文件，需使用此参数

  -f：解压缩时，若输出的文件与现有文件同名时，默认不会覆盖现有的文件。若要覆盖，可使用此选项

  -v：显示命令执行过程

  -L：列出压缩文件内容

  -t：测试压缩文件是否正常，但不对其做解压缩操作



1.3 Compression

1.3.4 bunzip2

- 举例说明：

  - ✓ 例一：

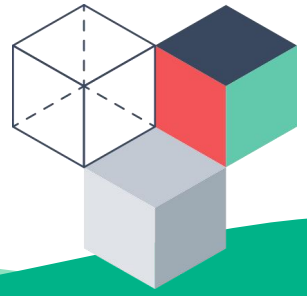
```
$ bunzip2 install.log.bz2
```

直接解压缩文件

  - ✓ 例二：

```
$ bunzip2 -r test/
```

解压test目录下的内容



1.3 Compression

1.3.5 zip

- “.zip”格式文件是 Windows 和 Linux 系统都通用的压缩文件类型
- zip 命令的基本格式如下：

  zip [选项] 压缩包名 源文件或源目录列表

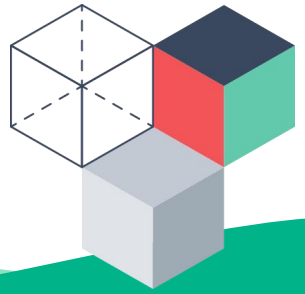
  备注：zip 压缩命令需要手工指定压缩之后的压缩包名，注意写清楚扩展名，以便解压缩时使用
- 参数说明：

  -r：递归压缩目录，即将制定目录下的所有文件以及子目录全部压缩

  -m：将文件压缩之后，删除原始文件，相当于把文件移到压缩文件中

  -v：显示详细的压缩过程信息

  -q　在压缩的时候不显示命令的执行过程



1.3 Compression

1.3.5 zip

-u：更新压缩文件，即往压缩文件中添加新文件

-压缩级别：压缩级别是从 1~9 的数字，-1 代表压缩速度更快，-9 代表压缩效果更好

- 举例说明：

✓ 例一：压缩单个文件

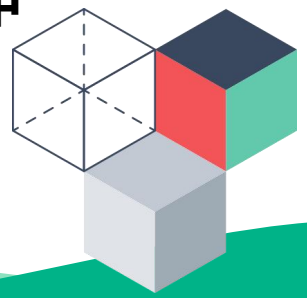
```
$ zip ana.zip anaconda-ks.cfg
```

压缩文件到ana.zip压缩包中

✓ 例二：压缩多个文件

```
$ zip test.zip install.log install.log.syslog
```

同时压缩多个文件到test.zip压缩包中



1.3 Compression

1.3.6 unzip

- unzip 命令可以查看和解压缩 zip 文件，该命令的基本格式如下：

  unzip [选项] 压缩包名

- 参数说明：

  -d 目录名：将压缩文件解压到指定目录

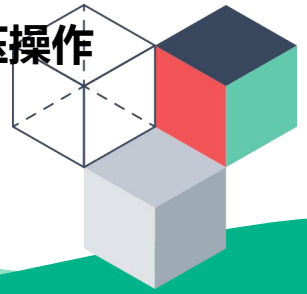
  -n：解压时并不覆盖已经存在的文件

  -o：解压时覆盖已经存在的文件，并且无需用户确认

  -v　查看压缩文件的详细信息，包括压缩文件中包含的文件大小、文件名以及压缩比等，但并不做解压操作

  -t：测试压缩文件有无损坏，但并不解压

  -x 文件列表：解压文件，但不包含文件列表中指定的文件



1.3 Compression

1.3.6 unzip

- 举例说明：
  - ✓ 例一：

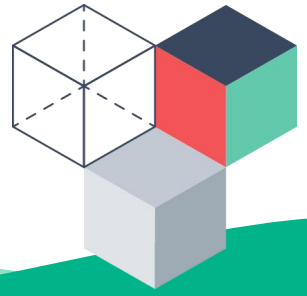
```
$ unzip dir1.zip
```

解压缩

  - ✓ 例二：

```
$ unzip -d /tmp/ ana.zip
```

把压缩包解压到指定位置



1.4 Archiving and compression

1.4.1 tar.gz and tar.bz2

- Linux 中，打包命令可以和压缩命令同时使用，该命令的基本格式如下：

  tar [选项] 压缩包 源文件或目录

  备注：此处常用的选项有以下 2 个，分别是：

  -z：压缩和解压缩 ".tar.gz" 格式

  -j：压缩和解压缩 ".tar.bz2"格式

  配合 tar 命令常用选项参数：

  -c：将多个文件或目录进行打包

  -x：对 tar 包做解打包操作

  -v：显示打包文件过程

  -f 包名：指定包的文件名

组合成4种常用选项参数：

解压缩：

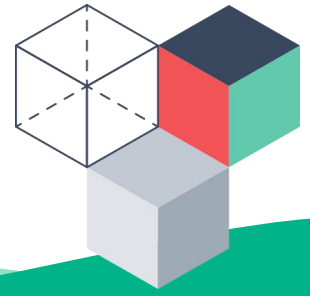
-jxvf

-zxvf

压缩：

-jcvf

-zcvf



1.4 Archiving and compression

1.4.1 tar.gz and tar.bz2

- L举例说明:

  ✓ 例一：压缩 ".tar.gz"格式

```
$ tar -zcvf tmp.tar.gz /tmp/
```

把/temp/目录直接打包压缩为".tar.gz"格式，

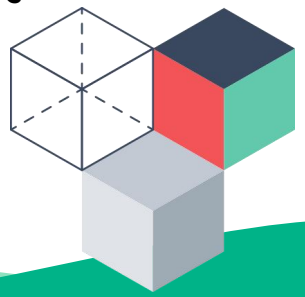
通过"-z"来识别格式，"-cvf"和打包选项一致

  ✓ 例二：压缩 ".tar.bz2"格式

```
$ tar -jcvf tmp.tar.bz2 /tmp/
```

把/temp/目录直接打包压缩为".tar.bz2"格式

通过"-j"来识别格式，"-cvf"和打包选项一致



1.4 Archiving and compression

1.4.1 tar.gz and tar.bz2

- 举例说明：

✓ 例三：解压缩 ".tar.gz"格式

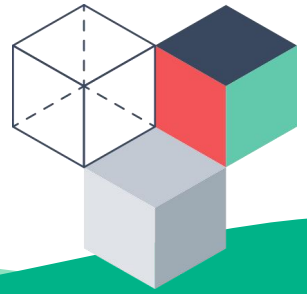
```
$ tar -zxvf tmp.tar.gz
```

解压缩tmp.tar.gz文件包（在线鸽子）

✓ 例四：解压缩 ".tar.bz2"格式

```
$ tar -jxvf tmp.tar.bz2
```

解压缩tmp.tar.bz2文件包（机械包子铺）



Post-assessment

1. Linux 为何要进行归档?
2. “tar.gz”格式文件用什么命令解压缩?
3. “tar.bz2”格式文件用什么命令压缩?

主观题 10分

设置

1. Linux为何要进行归档?
2. “tar.gz”格式文件用什么命令解压缩?
3. “tar.bz2”格式文件用什么命令压缩?

正常使用主观题需2.0以上版本雨课堂

作答

Summary

1. Archiving and Compression

a) 归档和压缩的联系与区别

b) tar命令用于打包和解打包

c) gzip、gunzip、bzip2、bunzip2、zip、unzip命令用于压缩和解压缩

d) tar+gzip、tar+bzip2进行打包及压缩 

Q&A

今日事今日毕

Question?



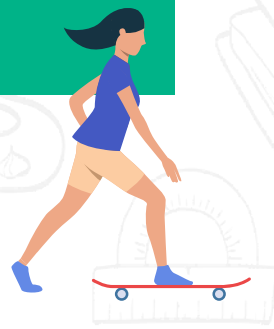
机器人操作系统（双语）

Robot Operating System (Bilingual)

WHERE THERE'S A WILL, THERE'S A WAY

授课教师：郄(qiè)龙飞

机器人工程



Overview lesson 3-3

1 Permission and Security

2 ~

3 ~

4 ~

Objectives

1、了解 Linux 中文件权限管理的方法

2、掌握变更文件访问权限的方法



Pre-assessment

1、Linux 中文件权限指什么?

2、Linux 中如何变更文件权限?



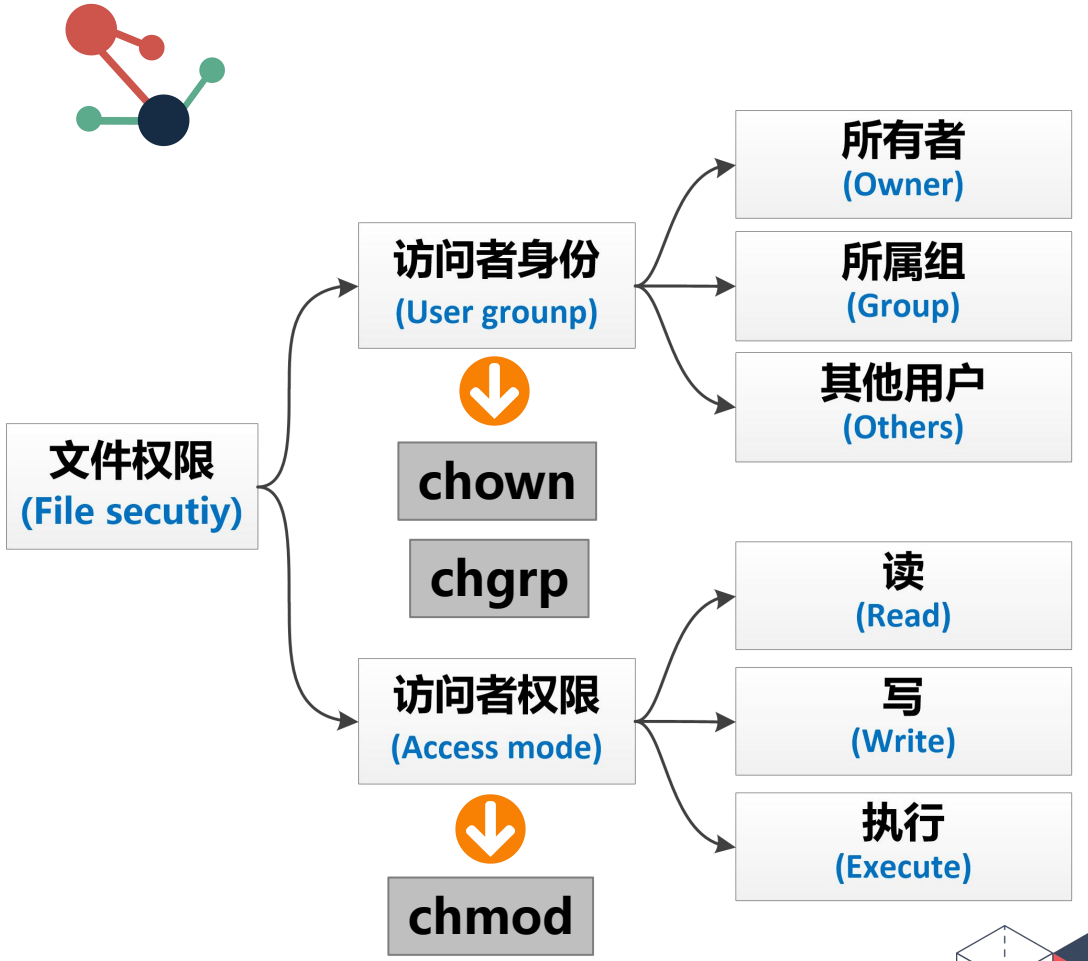
1

Permission and Security

1.1 Knowledge architecture

1.1.1 Overview

- 文件权限是指对文件的访问控制，决定哪些用户和哪些组对某文件(或目录)具有哪种访问权限
- Linux将文件访问者身份分为3个类别：所有者(user)、所属组(group)和其他用户(others)
- 对于每个文件，可以为这3类用户指定3种访问权限：读(read)、写(write)和执行(execute)
- 对文件权限的修改包括两个方面：修改文件所有者和用户对文件的访问权限

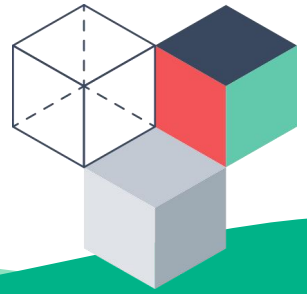


1.2 User group

1.2.1 Group identities

文件访问者身份是指文件权限设置所针对的用户和用户组，共有以下3种类型：

- **所有者：每个文件都有它的所有者，又称属主，简写为 u。默认情况下，文件的创建者即为其所有者。所有者对文件具有所有权，是一种特别权限**
- **所属组：指文件所有者所属的组，简称属组，简写为 g，可为该组指定访问权限。默认情况下，文件的创建者的主要组即为该文件的所属组**
- **其他用户：指文件所有者和所属组，以及 root 之外的所有用户，简写为 o。通常其他用户对文件拥有最低的权限，甚至没有权限**



1.3 Access mode

1.3.1 Access rights

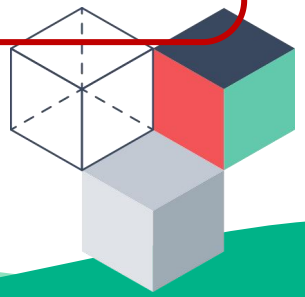
对于每个文件，针对上述3类身份的用户可指定以下3种不同级别的访问权限：

- 读：读取文件内容或者查看目录
- 写：修改文件内容或者创建、删除文件
- 执行：执行文件或者允许使用cd命令进入目录

文件访问者身份、访问权限都包括在文件属性中，使用ls -l(ll)命令显示文件详细信息

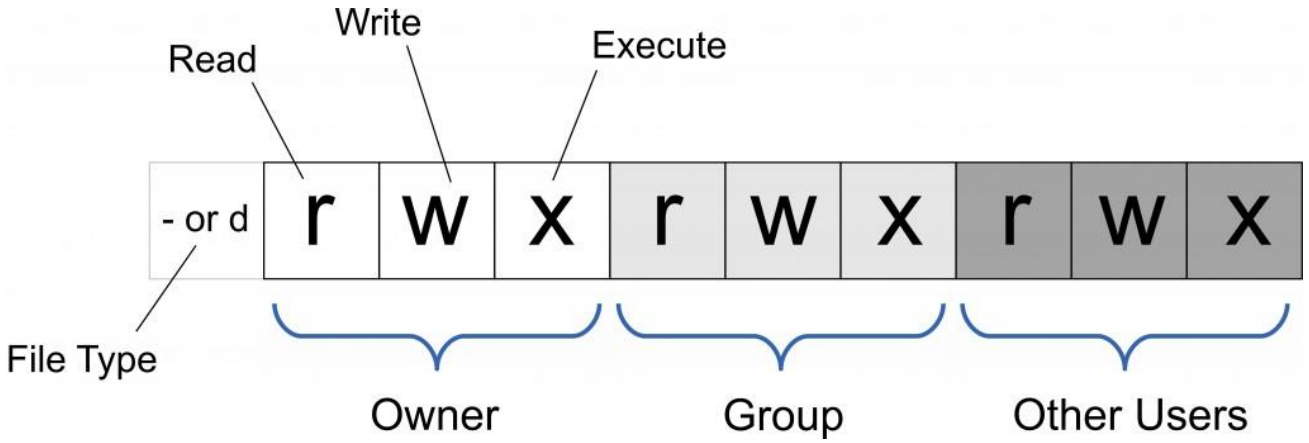
```
-rw-r--r--  1  zxp  zxp  8980  1月  5 11:37 examples.desktop
[文件权限] [链接] [所有者] [所属组] [容量] [  修改日期 ]   [ 文件名 ]
```

第 1 个字符表示文件类型，第 2 个字段表示该文件的链接数目，1 表示有一个硬链接，第 3 个字段表示这个文件的所有者，第 4 个字段表示这个文件的所属组，后面 3 个字段分别表示文件大小、修改日期和文件名称



1.3 Access mode

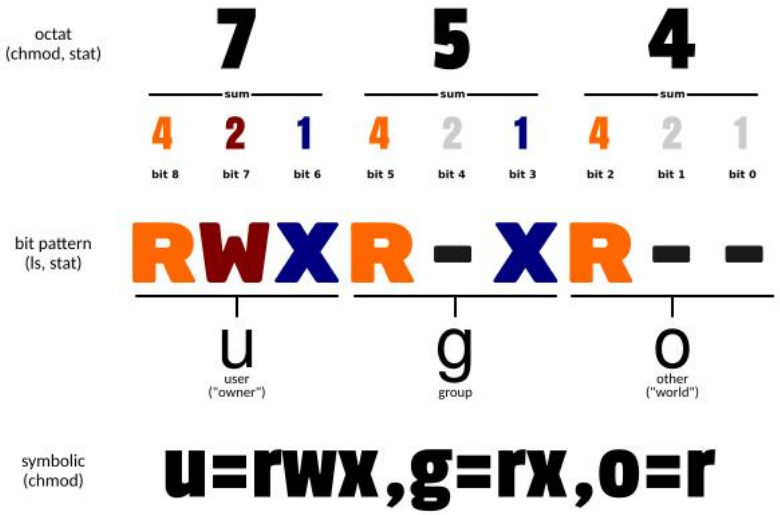
1.3.1 Access rights



- 第 1 个字符表示文件类型，d 表示目录文件，- 表示普通文件，l 表示链接文件，b 表示块设备文件，c 表示字符设备文件
- 2-10 字符分别表示文件所有者、所属组和其他用户的权限
- 3种文件权限依次用r、w 和 x 表示读、写和执行，如果没有某种权限，则在相应权限位置用-表示

1.3 Access mode

1.3.1 Access rights--Octal representation



权限	rwx	二进制	八进制
读 + 写 + 执行	rwx	111	7
读 + 写	rw-	110	6
读 + 执行	r-x	101	5
只读	r--	100	4
写 + 执行	-wx	011	3
只写	-w-	010	2
只执行	--x	001	1
无	---	000	0

- **读、写和执行可用8进制数字表示：**
  - **读：r --> 4**
  - **写：w --> 2**
  - **执行：x --> 1**

1.4 The tools

1.4.1 chown

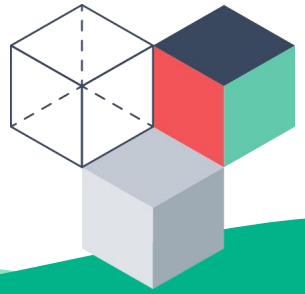
- 文件所有者可以变更，即将所有权转让给其他用户，只有root才有权变更所有者
- 使用 chown 命令变更文件所有者，使其他用户对文件具有所有权，基本用法如下：

  chown [选项] [新所有者] 文件列表

  备注：使用选项-R进行递归变更，即目录连同其子目录下的所有文件的所有者都变更
- 执行 chown 命令需要 root 权限，需要使用 sudo 命令
- 举例说明：

```
$ sudo chown wang news
```

将 news 的所有者改为 wang



1.4.2 chgrp

- 使用 chgrp 命令可以变更文件的所属组，基本用法如下：

  chgrp [选项] [新的所属组] 文件列表

- 亦可使用 chown 命令变更文件所属组，使其他用户对文件具有所有权，基本用法如下：

  chown [选项] [新所有者]:[新的所属组] 文件列表

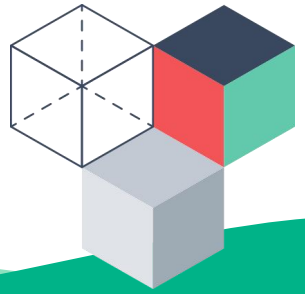
  备注：使用选项-R进行递归变更，即目录连同其子目录下的所有文件的所属组都变更

- 执行 chgrp 命令需要 root 权限，需要使用 sudo 命令

- 举例说明：

```
$ sudo chgrp shentong fenti
```

改变文件 fenti 所属组为 shentong



1.4 The tools

1.4.3 chmod

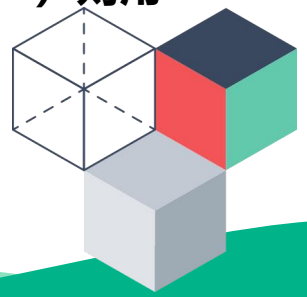
- root 和文件所有者可以修改文件访问权限，使用 chmod 命令来修改文件权限，基本用法如下：

  chmod [选项] 模式 文件名

- 模式是文件权限的表达式，有字符和8进制两种表示方法；
- 执行 chmod 命令需要 root 权限，需要使用 sudo 命令

1. 文件权限用字符表示：

   - “+”表示增加某种权限，”-”表示撤销某种权限，”=”表示指定某种权限（同时会取消其他权限）
   - 对于用户类型，所有者、所属组和其他用户分别用字符 u、g、o 表示，全部用户（包括3种用户）则用 a 表示
   - 权限类型用r（读）、w（写）和 x（执 行）表示



1.4.3 chmod

➢ 举例说明：

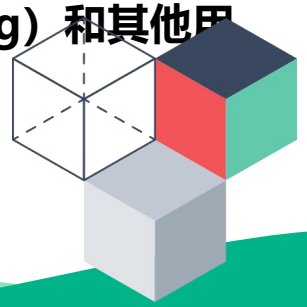
```
$ sudo chmod g+w, o+r /home/ros/myfile
```
给所属组用户增加写权限，给其他用户增加读权限

```
$ sudo chmod go-r /home/ros/myfile
```
同时撤销所属组和其他用户对该文件的读权限

```
$ sudo chmod a=rx /home/ros/myfile
```
对所有用户赋予读和执行权限

2. 文件权限用数字表示：

- 将权限读（r）、写（w）和执行（x）分别用数字 4、2 和 1 表示，没有任何权限则表示为 0
- 每一类用户的权限用其各项权限的和表示（结果为数字0~7）,依次为所有者（u）、所属 组（g）和其他用户（O）的权限



1.4.3 chmod

- 举例说明：

  - ✓ 例一：

    754 表示所有者、所属组和其他用户的权限依次为[4+2+1]、[4+0+1]、[4+0+0]，转化为字符表示就是：rwxr-xr--

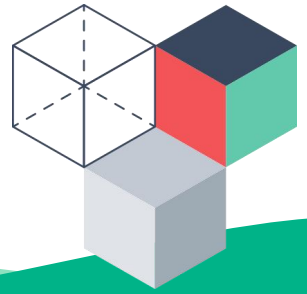
  - ✓ 例二：

    ```
    $ sudo chmod 644 file
    ```

    # 使文件 file 的所有者拥有读写权限，所属组用户和其他用户只能读取

    相当于：

    ```
    $ sudo chmod u=rw,go=r file
    ```



Post-assessment

1. 文件访问者身份有哪几种?
2. 文件权限有哪几种?
3. chmod、chgrp 和 chown的区别是什么?
4. ls -l 命令的输出结果中，第一个字符表示什么意思?

主观题 10分

设置

1. 文件访问者身份有哪几种?
2. 文件权限有哪几种?
3. chmod、chgrp 和 chown的区别是什么?
4. ls -l 命令的输出结果中，第一个字符表示什么意思?

正常使用主观题需2.0以上版本雨课堂

作答

ROS Summary

1. Permission and Security

a) 文件访问者 3 种身份、文件3种访问权限

b) u、g、o 与 w、r、x 的定义与使用 

c) chown、chgrp 变更文件访问者身份

d) chmod修改文件访问权限 

Homework



作业：学习《C++从入门到精通-作业版本》第3章

Q&A

今日事今日毕

Question?